

第四模块：DMA与高速数据通路（内核级超详细版 · Cortex-M 装备方向）

本模块目标：从“知道DMA能省CPU”升级到“理解数据在总线、DMA、缓存、内存之间如何真实流动”，并能设计稳定的高速数据通路。

本模块不讲“如何点HAL配置”，只讲底层机制与工程设计。

一、为什么需要DMA（从系统瓶颈讲清楚）

1.1 CPU搬运数据的代价

如果每个字节都由CPU搬运：

- 会消耗大量指令周期
- 会增加中断频率
- 会导致实时任务抖动

DMA的本质：

👉 让外设与内存之间的数据搬运不经过CPU执行流。

1.2 DMA解决的不是“速度”，是“确定性”

很多系统不是带宽不足，而是：

- CPU占用不可预测
- ISR抖动导致控制不稳定

DMA的价值：

让CPU负载更稳定。

二、DMA到底是什么（硬件视角）

DMA控制器是一个硬件状态机。

它做三件事：

- 1) 从源地址读
- 2) 写到目的地址
- 3) 计数减一

重复直到完成。

三、总线与仲裁（很多人从不考虑）

3.1 MCU内部不仅一个“通道”

典型系统有：

- CPU主端口
- DMA主端口
- 外设端口

它们共享总线资源。

3.2 仲裁导致的性能抖动

当CPU与DMA同时访问内存：

- 会产生总线争用
- 带来延迟抖动

工程结论：

高实时控制任务要避免与大DMA传输重叠。

四、DMA传输模式（必须掌握）

4.1 基本模式

- 内存到内存
 - 外设到内存
 - 内存到外设
-

4.2 循环模式（Circular）

用于连续数据流，如UART/ADC。

关键点：

DMA不会停止，而是循环写缓冲区。

必须设计：

- 读写指针
 - 半满/满中断处理
-

4.3 双缓冲（Double Buffer）

思想：

- Buffer A 接收
- Buffer B 处理

优点：

- 避免覆盖
 - 提高吞吐
-

五、DMA中断的正确使用方式

DMA中断通常包括：

- Half Transfer
- Transfer Complete
- Error

正确模型：

DMA ISR 只做：

- 清标志
- 记录事件
- 通知任务

禁止：

在ISR里做复杂解析/滤波。

六、缓存一致性（Cache Coherency）—— 高端必考点

说明：是否存在该问题取决于MCU是否有D-Cache。

6.1 为什么会出现“DMA写了，但CPU读不到”

如果CPU读缓冲区时命中D-Cache：

- CPU看到的是缓存旧数据
- DMA更新的是内存

结果：

CPU读不到DMA的新数据。

6.2 解决方案（必须会说清）

DMA接收后（外设→内存）：

- CPU读取前：Invalidate D-Cache

DMA发送前（内存→外设）：

- DMA读取前：Clean D-Cache

注意：

必须按缓存行对齐处理。

七、Memory Barrier 在DMA中的作用（很多人混淆）

区分：

- volatile：编译器层
- DMB/DSB：CPU执行顺序层

典型场景：

- 设置DMA寄存器后需要确保写入生效再启动
- DMA完成标志读取后需要确保内存可见

工程结论：

volatile 解决不了缓存/总线顺序问题。

八、零拷贝（Zero-Copy）数据通路设计

目标：

减少数据复制次数。

推荐结构：

DMA → RingBuffer → 解析任务（指针方式）

关键点：

- 缓冲区生命周期管理
- 不要把临时指针传给异步任务

九、高频底层考点（标准答案）

Q 1：DMA为什么能提升实时性？

标准答案：

DMA降低CPU搬运与中断频率，使CPU负载更稳定，从而提高实时性。

Q 2：为什么DMA循环模式必须配合半满/满中断？

标准答案：

因为DMA持续写入，必须用半满/满点作为处理时机，否则会覆盖未处理数据。

Q 3：为什么有缓存时DMA会出现读不到新数据？

标准答案：

CPU可能读的是D-Cache旧数据，DMA更新的是内存，需要做cache clean/invalidate。

Q 4：volatile能解决DMA一致性问题吗？

标准答案：

不能。volatile只约束编译器，不处理cache一致性与总线顺序。

十、工程注意事项（装备级经验）

- 1) DMA缓冲区尽量对齐到缓存行。
 - 2) DMA回调里不要做耗时处理。
 - 3) 高实时任务避免与大DMA传输重叠。
 - 4) 设计可观测：统计溢出、丢包、错误次数。
 - 5) 必须处理DMA Error中断。
-

十一、深度练习（强烈建议）

- 实现 UART DMA 循环接收 + RingBuffer + 帧解析。
 - 在有D-Cache的平台验证 clean/invalidate 的效果。
 - 故意制造DMA溢出并设计恢复策略。
-

完成本模块后，你将具备“高速数据通路设计能力”，这是装备类嵌入式系统的核心竞争力之一。